Università degli Studi di Torino

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

Tecnologie del Linguaggio Naturale

Lab 4 — Modellazione del Linguaggio con N-gram

Modeling social media and literary language with N-grams

Gruppo di lavoro:

Studente	**Matricola**
Alessandro Olivero	915069
Samuele Iudice	1235245
Andrea Franco	1226739

Docente: Prof. Daniele Radicioni

Anno Accademico: 2024/2025

Torino, 28 aprile 2026

Indice

1 Introduzione

L'obiettivo di questo laboratorio è modellare e confrontare due varietà linguistiche radicalmente diverse attraverso modelli N-gram: il linguaggio dei **social media** (Twitter) e il **linguaggio letterario** ottocentesco (*Emma* di Jane Austen, 1816).

L'intuizione di fondo è semplice: un tweet e una frase di un romanzo dell'Ottocento sono testi profondamente diversi, e questa diversità dovrebbe essere catturabile in modo quantitativo. I modelli N-gram, pur essendo tra gli strumenti più semplici del NLP, si prestano bene a questo tipo di analisi: contano le sequenze di parole più frequenti e ne stimano le probabilità, rivelando le strutture ricorrenti tipiche di ciascun dominio.

1.1 Obiettivi specifici

- Estrarre e confrontare gli N-gram (unigrammi, bigrammi, trigrammi) più frequenti nei due corpora, con e senza stopwords.
- Addestrare modelli di linguaggio MLE e Laplace e mostrarne differenze e limiti pratici.
- Analizzare la *perplexity* cross-domain e discuterne i limiti quando i vocabolari hanno dimensioni diverse.
- Implementare un classificatore basato su *log-likelihood* che superi i limiti della perplexity.
- Quantificare le differenze linguistiche attraverso il Type-Token Ratio (TTR) e la lunghezza media delle frasi.
- Generare testo nei due stili e confrontare qualitativamente i risultati.

1.2 Strumenti e librerie

- **Python 3.x** — linguaggio di implementazione
- **NLTK** — tokenizzazione, corpora, modelli N-gram
- **Matplotlib** — visualizzazione dei risultati
- **re** (regex) — pulizia dei testi

2 Dataset

2.1 Twitter Samples

Il corpus Twitter proviene dal pacchetto `nltk.corpus.twitter_samples`, che include tweet annotati automaticamente in base alla presenza di emoticon positive o negative, raccolti nel 2015.

- `positive_tweets.json` — 5.000 tweet con emoticon `:)`
- `negative_tweets.json` — 5.000 tweet con emoticon `:(`
- **Totale**: 10.000 tweet

Si è scelto di usare questo sottoinsieme invece del file `tweets.20150430-223406.json` (20.000 tweet generici), perché quest'ultimo era dominato da contenuti politici relativi alle elezioni britanniche del 30 aprile 2015 (menzionava Ed Miliband, David Cameron, SNP), rendendo il confronto meno significativo. I tweet positivi/negativi hanno invece un linguaggio più vario e rappresentativo del social media in generale.

```
from nltk.corpus import twitter_samples

tweets_pos = twitter_samples.strings('positive_tweets.json')
tweets_neg = twitter_samples.strings('negative_tweets.json')
tweets_raw = tweets_pos + tweets_neg  # 10.000 tweet totali
```

Listing 1: Caricamento del corpus Twitter

Esempi rappresentativi:

- *Positivo*: `#FollowFriday @France_Inte for being top engaged members this week :)`
- *Negativo*: `hopeless for tmr :(`

2.2 Corpus Letterario — Emma di Jane Austen

Il testo letterario è stato estratto dal corpus Gutenberg di NLTK, che raccoglie opere classiche di dominio pubblico. È stato scelto ***Emma* di Jane Austen (1816)** come rappresentante del linguaggio letterario formale ottocentesco: il romanzo è lungo, narrativamente ricco, e utilizza costruzioni sintattiche complesse con un vocabolario formale e preciso.

```
from nltk.corpus import gutenberg

testo_letterario_raw = gutenberg.raw('austen-emma.txt')
```

```
# 887.071 caratteri totali
```

Listing 2: Caricamento del corpus letterario

2.3 Statistiche comparative

La tabella seguente riassume le principali caratteristiche dei due corpora dopo il preprocessing.

Metrica	Twitter	Letterario
Numero di testi	10.000 tweet	1 romanzo
Token totali	94.706	158.267
Parole uniche	12.024	9.305
Lunghezza media	16,0 parole	25,6 parole
Type-Token Ratio	0,1270	0,0588

Tabella 1: Statistiche comparative dei due corpora dopo preprocessing

3 Preprocessing

Prima di costruire i modelli N-gram, i testi devono essere puliti e normalizzati. I due corpora richiedono trattamenti diversi.

3.1 Pulizia dei tweet

I tweet contengono elementi non linguistici che andrebbero a sporcare il modello: URL, menzioni utente (`@user`), simboli degli hashtag, emoticon e punteggiatura.

```
def pulisci_tweet(testo):
    testo = re.sub(r'http\S+', '', testo)       # rimuove URL
    testo = re.sub(r'@\w+', '', testo)           # rimuove @menzioni
    testo = re.sub(r'#', '', testo)              # rimuove simbolo #
    testo = re.sub(r'[^a-zA-Z\s]', '', testo)   # solo lettere
    testo = testo.lower().strip()
    return testo
```

Listing 3: Funzione di pulizia dei tweet

Esempio di trasformazione:

- **Prima:** `#FollowFriday @France_Inte for being top engaged :)`

- **Dopo:** `followfriday for being top engaged`

3.2 Pulizia del testo letterario

Per il romanzo è sufficiente rimuovere la punteggiatura e normalizzare il caso, dato che non contiene URL o menzioni.

```
def pulisci_letterario(testo):
    testo = re.sub(r'[^a-zA-Z\s]', '', testo)
    testo = testo.lower().strip()
    return testo
```

Listing 4: Funzione di pulizia del testo letterario

3.3 Tokenizzazione

La tokenizzazione converte la stringa di testo in una lista di token (parole). Si è usata la funzione `word_tokenize` di NLTK invece del semplice `split()`, perché gestisce correttamente le contrazioni inglesi e separa la punteggiatura residua.

Metodo	Output su "I can't stop,"
`split()`	`["I", "can't", "stop,"]`
`word_tokenize()`	`["I", "ca", "n't", "stop", ","]`

Tabella 2: Differenza tra split() e word_tokenize()

3.4 Rimozione delle stopwords

Le stopwords sono parole molto frequenti in qualsiasi testo inglese ("the", "a", "in", "of"...) e non sono discriminative tra i due domini. La loro rimozione permette di far emergere i bigrammi davvero caratteristici di ciascun corpus.

```
from nltk.corpus import stopwords

stop = set(stopwords.words('english'))
tokens_tweet_filtrati = [t for t in tokens_tweet if t not in
    stop]
tokens_lett_filtrati  = [t for t in tokens_lett  if t not in
    stop]
```

Listing 5: Rimozione delle stopwords

4 Analisi N-gram

4.1 Definizione

Un **N-gram** è una sequenza contigua di N parole in un testo. Dato un testo $w_1, w_2, \ldots, w_T$, gli N-gram di ordine N sono tutte le sequenze $(w_i, w_{i+1}, \ldots, w_{i+N-1})$ per $i = 1, \ldots, T - N + 1$.

- **Unigrammi** ($N = 1$): parole singole. Es.: "i love cats" → `[i, love, cats]`
- **Bigrammi** ($N = 2$): coppie consecutive. Es.: "i love cats" → `[(i,love), (love,cats)]`
- **Trigrammi** ($N = 3$): triplette consecutive. Es.: "i love cats" → `[(i,love,cats)]`

4.2 Bigrammi senza stopwords

La rimozione delle stopwords fa emergere le coppie di parole più caratteristiche di ciascun dominio.

Dominio	Bigramma	Frequenza
Twitter	(wan, na)	123
Twitter	(follow, please)	81
Twitter	(gon, na)	72
Twitter	(follow, u)	62
Twitter	(happy, birthday)	54
Letterario	(mr, knightley)	248
Letterario	(mrs, weston)	222
Letterario	(mr, elton)	184
Letterario	(miss, woodhouse)	148
Letterario	(frank, churchill)	123

Tabella 3: Top bigrammi per dominio dopo la rimozione delle stopwords

I bigrammi Twitter mostrano chiaramente le interazioni sociali tipiche della piattaforma: *wanna* e *gonna* (spezzati dal tokenizzatore in `wan na` e `gon na`), richieste di follow reciproco, auguri di compleanno. I bigrammi letterari sono quasi esclusivamente titoli e cognomi dei personaggi del romanzo, riflettendo la struttura narrativa del testo e il registro formale dell'inglese ottocentesco.

4.3 Bigrammi con stopwords

Dominio	Bigramma	Frequenza
Twitter	(thank, you)	260
Twitter	(i, love)	173
Twitter	(i, miss)	160
Twitter	(love, you)	155
Twitter	(i, cant)	155
Letterario	(to, be)	602
Letterario	(of, the)	563
Letterario	(i, am)	363
Letterario	(she, had)	322
Letterario	(had, been)	305

Tabella 4: Top bigrammi per dominio con stopwords

Alcune osservazioni interessanti:

- Twitter usa *cant* (contrazione senza apostrofo), mentre il romanzo usa *i am* (forma estesa): è una differenza di registro molto netta.
- La struttura *had been* (past perfect) è molto frequente nel romanzo (305 occorrenze) e praticamente assente nei tweet, dove i tempi verbali sono più semplici.
- Twitter è dominato da espressioni affettive dirette (*i love*, *love you*, *i miss*), tipiche della comunicazione personale sui social.

4.4 Trigrammi

Dominio	Trigramma	Frequenza
Twitter	(i, love, you)	84
Twitter	(you, so, much)	74
Twitter	(thanks, for, the)	67
Twitter	(i, want, to)	63
Letterario	(i, do, not)	123
Letterario	(i, am, sure)	97
Letterario	(she, could, not)	68
Letterario	(a, great, deal)	63

Tabella 5: Top trigrammi per dominio

I trigrammi letterari mostrano costruzioni tipiche del parlato ottocentesco trascritto: *i do not*, *i am sure*, *she could not*. Twitter al contrario produce espressioni emotive brevi e ripetitive: *i love you*, *you so much*.

5 Modelli di Linguaggio

5.1 Maximum Likelihood Estimation (MLE)

Il modello MLE stima la probabilità di una parola dato il contesto precedente semplicemente contando le occorrenze nel corpus di training. Per i bigrammi, la formula è:

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})} \tag{1}$$

dove $C(\cdot)$ indica il numero di occorrenze nel corpus.

```
from nltk.lm import MLE
from nltk.lm.preprocessing import padded_everygram_pipeline

n = 2  # bigrammi
train_data, vocab = padded_everygram_pipeline(n, [
    tokens_tweet])

modello_tweet = MLE(n)
modello_tweet.fit(train_data, vocab)
```

Listing 6: Addestramento del modello MLE

La funzione `padded_everygram_pipeline` aggiunge i simboli `<s>` (inizio frase) e `</s>` (fine frase) ai token e costruisce tutti gli N-gram fino all'ordine n richiesto.

5.1.1 Probabilità condizionali osservate

Parola	Contesto	P (Twitter)	P (Letterario)
love	i	0,0530	0,0017
wait	cant	0,1146	0,0000
happy	so	0,0138	—
mr	said	0,0000	0,0802
been	had	—	0,1885

Tabella 6: Probabilità condizionali MLE nei due modelli

I risultati confermano le aspettative. *love* segue *i* con probabilità del 5,3% su Twitter e quasi zero nel romanzo. *had been* è frequentissimo nel romanzo (18,9%), riflettendo l'uso narrativo del past perfect, mentre è praticamente assente nei tweet.

5.1.2 Il problema dello zero

Il principale limite del MLE è che assegna probabilità esattamente **zero** a qualsiasi sequenza non osservata durante il training. Questo causa perplexity infinita su qualsiasi testo che contenga anche una sola parola o coppia mai vista:

$$P_{\text{MLE}}(\text{“xyzword”} \mid \text{“i”}) = \frac{0}{C(\text{“i”})} = 0 \quad \Rightarrow \quad \text{PP} = \infty \tag{2}$$

```
print(modello_tweet.score('xyzword', ['i']))
# -> 0.000000

frase_rara = word_tokenize("i xyzword lol")
print(modello_tweet.perplexity(frase_rara))
# -> inf
```

Listing 7: Dimostrazione del problema dello zero

Questo è un problema reale: anche un solo token fuori vocabolario rende inutilizzabile la perplexity MLE come metrica di valutazione.

5.2 Smoothing di Laplace

Il **Laplace smoothing** (o add-one smoothing) risolve il problema aggiungendo 1 a tutti i conteggi, incluse le coppie mai osservate:

$$P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + |V|} \tag{3}$$

dove $|V|$ è la dimensione del vocabolario. In questo modo nessuna sequenza ha probabilità zero, ma le sequenze mai viste ricevono una probabilità molto piccola.

```
from nltk.lm import Laplace

modello_lp = Laplace(n)
modello_lp.fit(train_data, vocab)

print(modello_lp.score('xyzword', ['i']))
# -> 0.000065  (piccolo ma non zero)

```

```
print(modello_lp.perplexity(frase_rara))
# -> 1677.95  (alta ma non infinita)
```

Listing 8: Addestramento del modello Laplace

Metrica	MLE	Laplace
P(“xyzword” | “i”)	0,000000	0,000065
Perplexity su parola mai vista	∞	1677,95
Gestisce OOV[1]	No	Sì

Tabella 7: Confronto MLE vs Laplace

5.3 Scelta del valore di N

N	PP frase Twitter	PP frase Letterario
1	5170,14	12025,00
2	5170,91	12027,00
3	5170,92	12027,00

Tabella 8: Perplexity Laplace per $N = 1, 2, 3$ (modello Twitter)

I valori quasi identici per $N = 1, 2, 3$ sono sintomo di un problema di **data sparsity**: con soli 94.706 token Twitter, la maggior parte dei bigrammi e trigrammi possibili appare raramente o mai nel corpus. Laplace tratta quindi quasi tutte queste sequenze come sconosciute, producendo perplexity simili indipendentemente da N. Con un corpus più grande le differenze sarebbero più marcate, e i trigrammi catturerebbero strutture sintattiche più complesse.

6 Perplexity e Classificazione

6.1 Perplexity

La **perplexity** di un modello su una sequenza $W = w_1 \dots w_N$ misura quanto il modello è “sorpreso” da quel testo:

$$\mathrm{PP}(W) = P(w_1, \dots, w_N)^{-\frac{1}{N}} = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i \mid w_{i-1})}} \tag{4}$$

Un valore **basso** significa che il modello assegna alta probabilità alla sequenza (la conosce bene); un valore **alto** indica sorpresa.

[1] OOV = Out-Of-Vocabulary, parole non presenti nel training set.

6.2 Limiti della perplexity per la classificazione

Frase	Atteso	PP Twitter	PP Lett.	Risultato
"i love you so much"	TW	5170,9	4488,5	TW ✓
"so happy right now"	TW	12027,0	9308,0	LT ×
"i miss you feel sad"	TW	5746,3	4917,0	TW ✓
"she had been very well"	LT	12027,0	9308,0	TW ×
"it was a very good"	LT	6756,2	5258,0	TW ×

Tabella 9: Classificazione con perplexity Laplace: accuratezza 3/5

Il classificatore basato su perplexity è inaffidabile. Il motivo è strutturale: il modello Twitter ha un vocabolario più grande (12.027 parole) rispetto a quello letterario (9.308 parole). Con lo smoothing di Laplace, un vocabolario più grande "diluisce" la probabilità su più termini, producendo sistematicamente perplexity più alta. Il classificatore quindi tende sempre a preferire il modello con vocabolario più piccolo, indipendentemente dal contenuto del testo.

6.3 Classificatore con Log-Likelihood

Per superare questo limite si è adottata la **log-likelihood** come metrica di classificazione:

$$\mathcal{L}(W) = \log P(w_1, \ldots, w_N) = \sum_{i=2}^{N} \log P(w_i \mid w_{i-1}) \tag{5}$$

Un valore **meno negativo** (più vicino a zero) indica che il modello assegna alta probabilità alla sequenza, ovvero la "conosce bene". Poiché la log-likelihood non dipende dalla dimensione del vocabolario in modo diretto come la perplexity, è una metrica più robusta per confrontare modelli su vocabolari di dimensioni diverse.

```python
import math

def log_likelihood(modello, tokens):
    score = 0.0
    for i in range(1, len(tokens)):
        prob = modello.score(tokens[i], [tokens[i-1]])
        score += math.log(prob) if prob > 0 else math.log(1e-10)
    return score

def classifica(testo, mod_tw, mod_lt):
```

```
    tokens = word_tokenize(testo.lower())
    ll_tw   = log_likelihood(mod_tw, tokens)
    ll_lett = log_likelihood(mod_lt, tokens)
    return "TWITTER" if ll_tw > ll_lett else "LETTERARIO"
```

Listing 9: Implementazione del classificatore log-likelihood

6.3.1 Risultati

Testo	LL Twitter	LL Lett.	Verdetto
"i love you so much thank you"	$-32,02$	$-42,15$	TWITTER ✓
"she had been very well disposed"	$-124,74$	$-21,91$	LETTERARIO ✓
"i miss you so much cant stop thinking"	$-41,54$	$-122,88$	TWITTER ✓
"it was a very good thing to have done"	$-29,21$	$-24,75$	LETTERARIO ✓
"happy birthday love you so much"	$-11,27$	$-58,54$	TWITTER ✓
"she was not a woman of many words"	$-70,82$	$-47,21$	LETTERARIO ✓
"i cant stop smiling so happy"	$-57,95$	$-97,56$	TWITTER ✓
"mr knightley had always been very fond"	$-108,42$	$-24,91$	LETTERARIO ✓

Tabella 10: Risultati classificatore log-likelihood: 8/8 corretti

Il classificatore log-likelihood classifica correttamente tutti gli 8 casi di test, rispetto ai 3/8 del classificatore basato su perplexity. Le differenze di log-likelihood sono particolarmente marcate per testi fortemente tipici di un dominio: "she had been very well disposed" ha LL letterario di $-21,91$ contro $-124,74$ per Twitter, una differenza di oltre 100 unità che riflette quanto quella struttura sintattica sia rara nei tweet.

7 Generazione del Testo

I modelli MLE possono essere usati per **generare** testo nello stile del corpus su cui sono stati addestrati. La generazione è un processo probabilistico: ad ogni passo, la parola successiva viene campionata dalla distribuzione condizionale data la parola precedente.

Si è scelto di usare MLE (e non Laplace) per la generazione, perché Laplace distribuisce la probabilità su tutti i token del vocabolario inclusi quelli sconosciuti, producendo sequenze di lettere singole invece di parole reali.

```
def genera_testo(modello, n_parole=40):
    testo_raw = modello.generate(n_parole, random_seed=42)
    # Filtra token speciali e artefatti
```

```
    testo_filtrato = [
        t for t in testo_raw
        if t not in ['<s>', '</s>', '<UNK>']
        and not (len(t) == 1 and t not in ['i', 'a'])
    ]
    return ' '.join(testo_filtrato)
```

Listing 10: Generazione del testo

Stile	Testo generato (seed=42)
Twitter	"papa coach it from the rain why are not a fantastic spring fiesta make sure the donation manager we all the rest his cute when im being a small polka dots"
Letterario	"over a good exchange consent to the cardroom looking about her less allied with persevering attention could not that a substance of her but yet heard any body to me"

Tabella 11: Esempi di testo generato con i due modelli (seed=42)

Il testo Twitter è frammentato e colloquiale, con espressioni informali (*im*, *polka dots*). Il testo letterario usa strutture più elaborate (*persevering attention*, *allied with*) tipiche della prosa ottocentesca.

Seed	Twitter	Letterario
99	"im getting bigger at the epic i love you in my life going"	"i can believe i shall decidedly in the neighbourhood who"
7	"have a new cards in march free insurance co i beleve"	"hartfield and return among their getting at last ah my"
123	"and by my bestfriend she brought us i want corn chips"	"and away from being to announce cheerful exulting than"

Tabella 12: Generazione con diversi seed casuali

8 Analisi Linguistica

8.1 Type-Token Ratio

Il **Type-Token Ratio** (TTR) misura la ricchezza lessicale di un corpus come rapporto tra tipi (parole distinte) e token (parole totali):

$$\text{TTR} = \frac{|\{w : w \in \text{corpus}\}|}{|\text{corpus}|} = \frac{\text{types}}{\text{tokens}} \tag{6}$$

Corpus	Types	Tokens	TTR
Twitter	12.024	94.706	0,1270
Letterario	9.305	158.267	0,0588

Tabella 13: Type-Token Ratio dei due corpora

Il TTR di Twitter è più del doppio rispetto al romanzo. Questo risultato è controintuitivo: ci si aspetterebbe che un romanzo ottocentesco abbia un vocabolario più ricco di tweet. La spiegazione sta nella natura eterogenea dei tweet: ogni tweet introduce hashtag, abbreviazioni, slang e nomi propri unici (`followfriday`, `unfollowers`, `bestfriend`), molti dei quali appaiono raramente nel corpus. Il romanzo invece ripete sistematicamente gli stessi personaggi e costruzioni grammaticali, abbassando il rapporto types/tokens.

8.2 Lunghezza media delle frasi

Corpus	Lunghezza media (parole)
Tweet	16,0
Frase letteraria	25,6

Tabella 14: Lunghezza media delle frasi nei due corpora

Le frasi letterarie sono in media 9,6 parole più lunghe dei tweet. Questo riflette la struttura sintattica complessa del romanzo ottocentesco, con proposizioni subordinate multiple, incisi e descrizioni elaborate, contrapposta alla brevità imposta dal limite di 140 caratteri di Twitter (al momento della raccolta del dataset nel 2015).

9 Limiti, Vantaggi e Confronto

9.1 Vantaggi degli N-gram

- **Semplicità e trasparenza**: l'intero modello si riduce a conteggi di sequenze. Le probabilità condizionali sono direttamente leggibili e interpretabili.
- **Efficienza computazionale**: addestramento e inferenza sono molto veloci, non richiedono GPU né risorse elevate.
- **Nessun training pre-esistente richiesto**: si addestrare da zero su qualsiasi corpus in pochi secondi.

9.2 Limiti degli N-gram

- **Data sparsity**: con dataset piccoli, la maggior parte dei bigrammi e trigrammi possibili non viene mai osservata nel training. Nel nostro caso, i 94.706 token Twitter sono insufficienti per stimare bene le probabilità di trigrammi, come dimostrano i valori di perplexity quasi identici per $N = 1, 2, 3$.
- **Contesto limitato**: i bigrammi considerano solo la parola immediatamente precedente. Non possono catturare dipendenze a lungo raggio come in "The man who I met yesterday **was** kind", dove *was* dipende da *man* che è lontano.
- **Nessuna semantica**: il modello non capisce il significato delle parole. *happy* e *joyful* sono token completamente diversi, anche se sinonimi.
- **Perplexity non comparabile tra vocabolari diversi**: come dimostrato nella sezione sulla classificazione, la perplexity con Laplace non è direttamente comparabile tra modelli con vocabolari di dimensioni diverse. È stato necessario ricorrere alla log-likelihood per ottenere un classificatore affidabile.

9.3 Confronto con approcci alternativi

Approccio	**Vantaggi**	**Svantaggi**	**Complessità**
N-gram	Semplice, veloce, interpretabile	Sparsità, no semantica, contesto corto	Bassa
Word2Vec	Semantica, vettori densi	No probabilità, no generazione	Media
LSTM / RNN	Contesto lungo, genera testo	Richiede molto training	Alta
Transformer	Stato dell'arte	Computazionalmente costoso	Molto alta

Tabella 15: Confronto tra approcci per la modellazione del linguaggio

10 Risultati e Conclusioni

10.1 Riepilogo

Esperimento	Risultato	Interpretazione
Bigrammi Twitter	(wan,na), (i,love), (i,cant)	Slang, espressioni emotive
Bigrammi Letterario	(mr,knightley), (had,been)	Titoli formali, past perfect
TTR Twitter	0,1270	Vocabolario eterogeneo
TTR Letterario	0,0588	Vocabolario ripetitivo
Lunghezza frasi TW	16,0 parole	Brevi, imposte dal limite 140 car.
Lunghezza frasi LT	25,6 parole	Complesse, subordinate multiple
Classif. perplexity	3/8 = 38%	Inaffidabile con vocabolari diversi
Classif. log-lik.	8/8 = 100%	Robusto, non dipende da $\|V\|$

Tabella 16: Riepilogo dei principali risultati

10.2 Conclusioni

Questo laboratorio ha mostrato che i modelli N-gram, pur nella loro semplicità, sono strumenti efficaci per caratterizzare e distinguere stili linguistici diversi.

I principali risultati ottenuti sono:

1. I bigrammi e trigrammi più frequenti rispecchiano fedelmente le caratteristiche culturali e stilistiche di ciascun dominio: slang e interazioni sociali per Twitter, titoli formali e strutture narrative complesse per il romanzo.

2. Il TTR più alto di Twitter è controintuitivo ma spiegabile: la varietà lessicale dei tweet deriva dall'eterogeneità dei contenuti e dall'uso di hashtag e abbreviazioni uniche, non da un vocabolario intrinsecamente più ricco.

3. La perplexity con Laplace non è una metrica affidabile per classificare testi tra modelli con vocabolari di dimensioni diverse. La log-likelihood è una metrica più robusta per questo scopo, raggiungendo il 100% di accuratezza sui casi di test.

4. I limiti osservati (sparsità, perplexity identica per $N = 1, 2, 3$) sono in gran parte attribuibili alla dimensione ridotta del corpus Twitter. Un dataset più grande ridurrebbe la sparsità e farebbe emergere differenze più nette tra unigrammi, bigrammi e trigrammi.

Sviluppi futuri potrebbero includere: l'uso di dataset più grandi, l'adozione di smoothing più sofisticati come Kneser-Ney, e il confronto con modelli neurali (LSTM, Transformer) per valutare il guadagno in qualità rispetto alla complessità computazionale.

Dichiarazione sull'uso di strumenti generativi

Come richiesto dalle linee guida del laboratorio, si dichiara che nella realizzazione di questo lavoro sono stati utilizzati strumenti di intelligenza artificiale generativa (Claude, Anthropic) per:

- Supporto nella strutturazione e nel debugging del codice Python.
- Identificazione e correzione di errori nel classificatore e nella generazione del testo.
- Formattazione della presente relazione in LaTeX.

Tutte le scelte progettuali, l'analisi e l'interpretazione dei risultati, e le conclusioni sono state elaborate autonomamente dal gruppo.